SENTINELX: CYBER ATTACK INTELLIGENCE AND THREAT HUNTING PLATFORM

PROJECT REPORT

Submitted by

Shlok Gupta (24BCI0061)

Shreya Bhushan (24BCI0086)

Yashneil (24BDS0158)

Prepared For

DATABASE SYSTEMS (BCSE302P) - Lab PROJECT

FACULTY:

Dr. Anand Bihari

School of Computer Science and Engineering

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

Abstract

Most organisations already collect a lot of security data, such as firewall logs, intrusion alerts and lists of known bad IP addresses from threat feeds. The problem is that this data sits in different places, and news about a bad IP usually arrives days or weeks after that IP has already been probing the network. This project is our attempt at a small Security Operations Centre (SOC) built around one relational database, so that all of this is stored once, linked properly and searched with plain SQL.

SentinelX keeps network logs, detected attacks, the servers they hit, attacker groups, malware, known vulnerabilities (CVEs), indicators of compromise, incidents and analysts in 16 MySQL tables, all in Third Normal Form (3NF). Each table keeps only the attributes that a feature actually uses. Apart from the usual features like a dashboard, attack search and incident handling, we added two ideas of our own. The first is a retro-hunt. When a new bad IP is added, the database goes back through the old logs on its own to check whether that IP already reached us before anyone knew it was bad. The second is a "time machine" for threat data. Every indicator keeps two time periods, when it was actually hostile and when we found out, so any search can be repeated using only what we knew on an earlier day. We tested the system on 600,340 generated log rows and 115,376 attacks.

Table of Contents

1. Introduction..... 4
2. Objectives..... 4
3. Project Scope..... 4
4. Key Contacts and Stakeholders..... 5
5. Project Resource Requirements..... 5
6. System Architecture..... 5
7. ER Diagram..... 6
8. Database Design, Tables and Constraints..... 10
9. Module-wise Functional Description..... 18
10. Work Break Down..... 19
11. Limitations and Future Scope..... 19
12. Conclusion..... 20

1. Introduction

Anyone who has seen how a small security team works knows the routine. An alert fires, someone opens the firewall console, then the IDS, then a spreadsheet of known bad IPs, and tries to match timestamps by hand. Each tool keeps its own copy of the facts, so the same attacker or server shows up in several places with slightly different details. Threat feeds also tend to be late. By the time an IP address appears on a blocklist it has often been scanning people for weeks, and nobody goes back to check last month's logs.

We picked this topic for our DBMS lab project because it fits a relational database very naturally. There are clear entities (servers, attackers, malware, vulnerabilities, incidents), clear relationships between them (an attack hits one server, an incident can group many attacks) and good reasons to use constraints, triggers, transactions and indexes. Cloud services like Amazon GuardDuty already cover a lot of this, but checking old logs for a newly reported bad IP is still a manual job there. We wanted the database to do that part by itself, and that became the main idea of the project.

2. Objectives

Broadly, we set out to:

- Design a normalised relational schema in Third Normal Form (3NF) for a security operations centre that avoids update, insert and delete problems.
- Draw a clean ER diagram that keeps only the attributes we really use, and make sure it matches the tables we actually built.
- Keep every column only as large as it needs to be (for example 50 characters for an email) and back every rule with a constraint.
- Let the database do routine work by itself, like opening incidents for serious attacks and checking old logs when a new bad IP is added.
- Build a simple web dashboard on top of the database so the design can be shown working with real queries.

3. Project Scope

Within the time we had, the system covers:

- Basic data about servers and devices (assets), CVEs, attack types mapped to MITRE ATT&CK, attacker groups and malware.
- Storing raw network logs and detected attacks, with indexes on time and source IP so searches over the large tables stay fast.
- A dashboard with attacks and blocked attacks in the last 24 hours, open critical incidents, malicious IPs and servers under attack.
- Searching attacks by IP, country, attack type, malware, CVE, severity and date.
- Incident handling: automatic creation, assigning an analyst, response steps and closing.
- Retro-hunting and the time machine described above.
- User roles (admin, analyst, viewer) with different permissions.

4. Key Contacts and Stakeholders

Name	Registration Number	Role	Email / Phone
Shlok Gupta	24BCI0061	Team Member	
Shreya Bhushan	24BCI0086	Team Member	
Yashneil	24BDS0158	Team Member	
Dr. Anand Bihari		Faculty Guide	

5. Project Resource Requirements

Hardware Requirements

- A laptop or desktop with an Intel i5 (8th gen) or Ryzen 5 and at least 8 GB RAM. 16 GB was more comfortable with the database, backend and frontend running together.
- About 3 GB of free disk space for the database, tools and packages.
- An internet connection for installing packages.

Software Requirements

- MySQL 8.0 (8.0.16 or later, which enforces CHECK constraints), with the mysql client and MySQL Workbench to work with it.
- Python 3.11 with FastAPI for the backend, and pandas and scikit-learn for test data and the ML parts.
- Node.js 20 with React and Vite for the dashboard.
- draw.io for planning the ER diagram.
- Docker to run the whole setup with one command.
- Visual Studio Code, Git and a shared GitHub repository.

6. System Architecture

We kept a simple layered setup with the database in the middle. Every other part either writes to it or reads from it. Logs are loaded in batches and each batch goes in as one transaction, so a failed batch leaves nothing half written. The rules that must never be skipped, like opening incidents and running retro-hunts, are triggers inside MySQL instead of application code. The FastAPI backend connects with a limited database user and serves the React dashboard, and the ML jobs read from the database and write their results back into it.

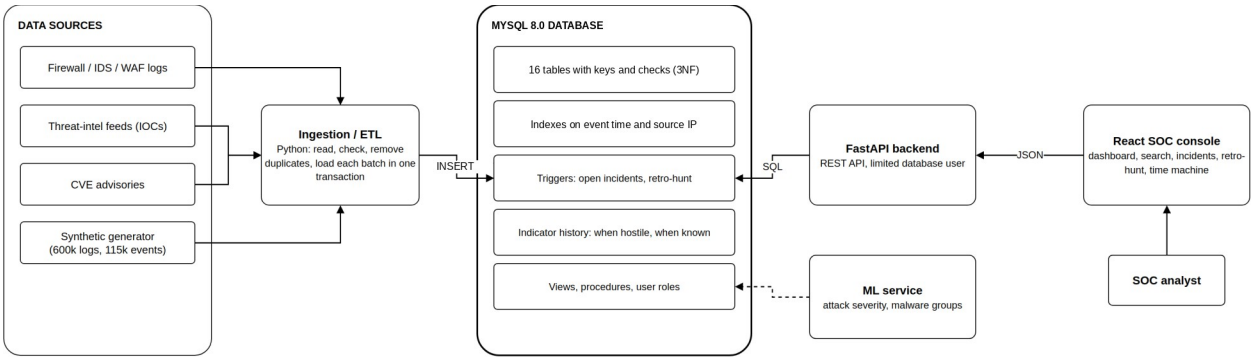


Fig. 1 System architecture

7. ER Diagram

Before looking at the diagram, here is what each entity means in this system:

Entity	What it represents
ASSET	A server or device we are protecting
VULNERABILITY	A published weakness (CVE) with its CVSS score
NETWORK_LOG	One raw line from the firewall or IDS
ATTACK_EVENT	One detected attack against an asset
ATTACK_TYPE	A kind of attack, mapped to a MITRE ATT&CK technique
MITIGATION (weak)	One response step for an attack type
THREAT_ACTOR	An attacker group
MALWARE	A piece of malicious software
IOC	An indicator of compromise, such as a bad IP, domain, file hash or URL
RETRO_HUNT_RUN	One automatic search of old logs for a newly added bad IP
INCIDENT	An investigation that groups related attacks
ANALYST	A person who uses the system

The ER diagram in Fig. 2 is drawn in Chen notation. Rectangles are entities, diamonds are relationships and ovals are attributes, with the primary key underlined. MITIGATION is a double rectangle because it is a weak entity: a step like "step 2" only makes sense inside its attack type, so its key is type_id plus step_no. Its partial key step_no has a dashed underline, and its relationship HAS_STEP is drawn as a double diamond. The ovals detected_at and patched_at hang off the EXPOSED_TO diamond because they describe the link between an asset and a CVE, not either one alone. A double line means every row on that side has to take part, for example every attack must target an asset. Each entity shows its key and at most three main attributes. Every column is listed in section 8. Foreign keys are not drawn as ovals because the relationship lines already show them.

Compared with our first version, the diagram now has no crossing lines and no long lines wrapping around the page, and 23 attributes that no feature used were removed (listed in section 8). It is easiest to read from the middle. ATTACK_EVENT is one detected attack. The triangle at the top is who attacks: MALWARE, THREAT_ACTOR and the IOCs that give them away. On the left is the attack catalogue: ATTACK_TYPE and

its MITIGATION steps. At the bottom right is what we protect: the ASSET an attack hits, the VULNERABILITY it used and the raw NETWORK_LOG lines sent to that asset. On the right is how we respond: an IOC triggers a RETRO_HUNT_RUN, which searches old logs and may open an INCIDENT that an ANALYST handles. There are 12 entities and 17 relationships in total. Twelve relationships are one-to-many (one of them identifying), four are many-to-many and one (OPENS) is one-to-one.

Relationship	Between	Type	What it means	Linked by
TARGETS	ASSET, ATTACK_EVENT	1 : N	An asset can be attacked many times, but each attack hits exactly one asset	attack_event.asset_id (NOT NULL)
CLASSIFIED_AS	ATTACK_TYPE, ATTACK_EVENT	1 : N	Each attack has exactly one type	attack_event.type_id (NOT NULL)
ATTRIBUTED_TO	THREAT_ACTOR, ATTACK_EVENT	1 : N	An attack may be linked to one attacker group	attack_event.actor_id
DELIVERS	MALWARE, ATTACK_EVENT	1 : N	An attack may carry one piece of malware	attack_event.malware_id
EXPLOITS	VULNERABILITY, ATTACK_EVENT	1 : N	An attack may use one CVE	attack_event.cve_id
EXPOSED_TO	ASSET, VULNERABILITY	M : N	An asset can have many CVEs and a CVE can affect many assets	link table asset_vulnerability
DESTINED_FOR	ASSET, NETWORK_LOG	1 : N	Each log line is sent to one asset	network_log.dst_asset_id (NOT NULL)
HAS_STEP	ATTACK_TYPE, MITIGATION	1 : N (identifying)	An attack type has numbered response steps	mitigation key (type_id, step_no)
DEVELOPS	THREAT_ACTOR, MALWARE	1 : N	A group can build many pieces of malware	malware.actor_id
LINKED_TO	THREAT_ACTOR, IOC	1 : N	An indicator may belong to one group	ioc.actor_id
INDICATES	MALWARE, IOC	1 : N	An indicator may point to one piece of malware	ioc.malware_id
MATCHES	ATTACK_EVENT, IOC	M : N	An attack can match many indicators and the other way round	link table event_ioc
TRIGGERS	IOC, RETRO_HUNT_RUN	1 : N	Every retro-hunt is started by one indicator	retro_hunt_run.ioc_version_id (NOT NULL)
HITS	RETRO_HUNT_RUN, NETWORK_LOG	M : N	A retro-hunt finds many old log lines	link table retro_hit
OPENS	RETRO_HUNT_RUN, INCIDENT	1 : 1	A retro-hunt opens at most one incident	retro_hunt_run.incident_id (UNIQUE)
GROUPS	INCIDENT, ATTACK_EVENT	M : N	An incident covers many attacks	link table incident_event
ASSIGNED_TO	ANALYST, INCIDENT	1 : N	An analyst handles many incidents	incident.assigned_to

Each kind of relationship becomes tables in a fixed way. A one-to-many relationship becomes a foreign key on the N side; if participation is total (a double line), that column is NOT NULL. A many-to-many

relationship becomes a link table whose primary key joins both keys. The one-to-one OPENS is a foreign key with a UNIQUE constraint. The identifying relationship HAS_STEP puts the owner's key inside the weak entity's primary key.

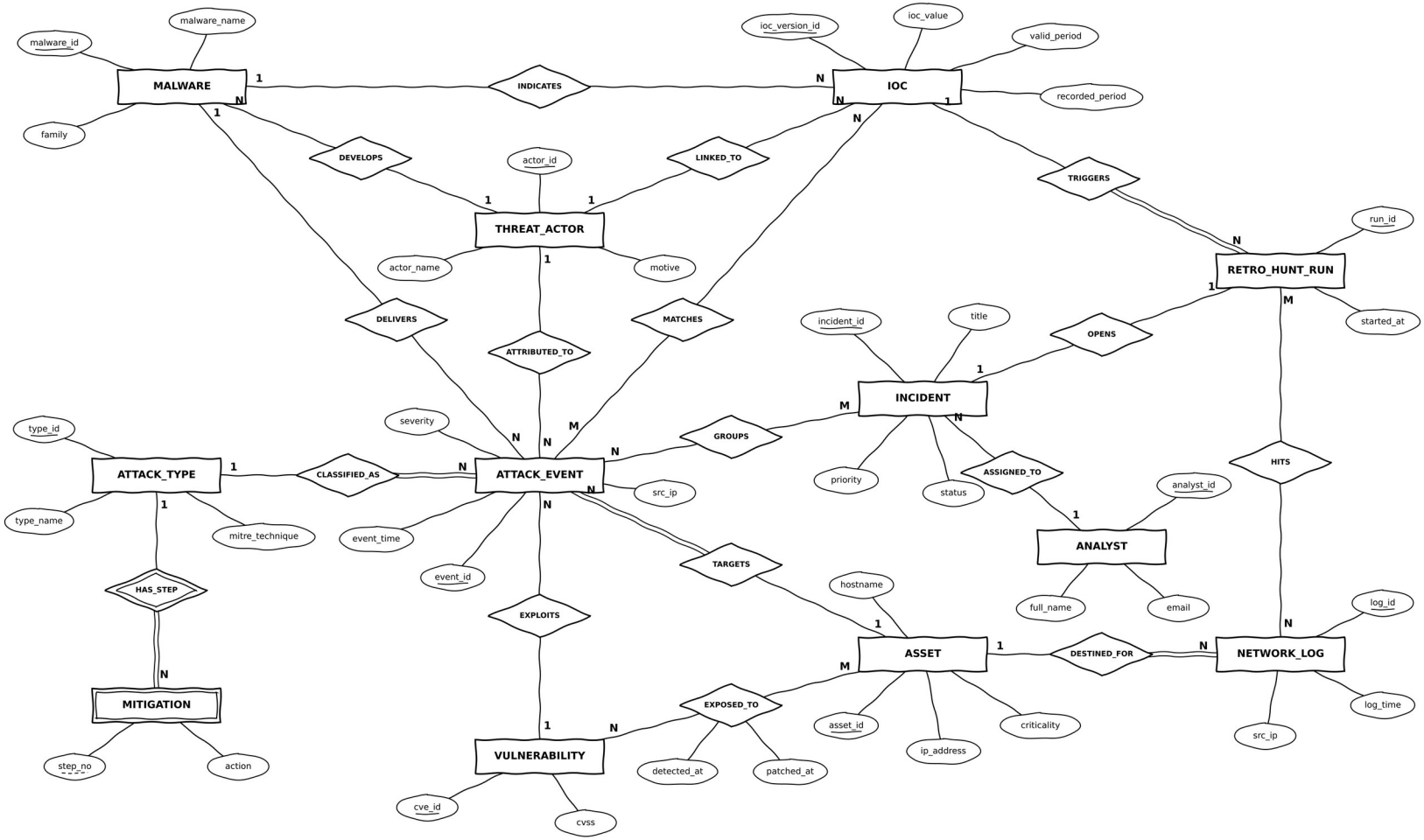


Fig. 2 ER diagram of SentinelX (Chen notation)

8. Database Design, Tables and Constraints

All 16 tables are in Third Normal Form (3NF). In every table, each non-key column depends on the key, the whole key and nothing but the key. In practice this means an attacker, a server or a CVE is stored exactly once. Changing an attacker's details touches one row, a server can be added before it is ever attacked, and deleting an attack never deletes what we know about the attacker. Every column that decides another column is itself a candidate key, so the tables also meet BCNF.

How we normalised

Step	What we did
Starting point	One flat row per attack with the time, source IP, host name, host OS, attack name, attacker name and country, malware name and family, CVE and score, and a list of IOCs like "198.51.100.23, evil-1.example".
1NF	Split the IOC list so each row holds one indicator. The key became (event, indicator).
2NF	Attack details depend only on the event and indicator details only on the indicator, so they went into ATTACK_EVENT, IOC and the link table EVENT_IOC.
3NF	Host details depend on the host, attacker details on the attacker, CVSS on the CVE and so on. Each of these moved to its own table: ASSET, THREAT_ACTOR, MALWARE, VULNERABILITY, ATTACK_TYPE and ANALYST.
3NF check	Checking every table again, we found two transitive dependencies in our own first draft. vulnerability.severity depended on cvss, not on cve_id, and malware.category depended on family, not on malware_id. Both columns were removed; a view works out severity from cvss.
Derived data	retro_hunt_run stored logs_matched and assets_touched, which are only counts of rows in retro_hit. We removed them and a view counts them instead, so they can never disagree with the hits.

Attributes we removed

After review we kept only the attributes that a module reads or writes. The 23 removed attributes are:

Entity	Removed	Why
ASSET	os, department, country_code	No page or query used them
VULNERABILITY	vendor, product, description, severity	Descriptive text is in the NVD; severity depends on cvss
NETWORK_LOG	bytes_sent, sensor	Not used by search, retro-hunt or the dashboard
ATTACK_TYPE	mitre_tactic	Reference data from the MITRE catalogue; never queried
THREAT_ACTOR	sophistication, first_seen	Not shown or filtered anywhere
MALWARE	category, severity, first_seen	category depends on family; severity belongs to the attack
IOC	confidence, source	Not used by retro-hunt or the time machine
RETRO_HUNT_RUN	finished_at, logs_matched, assets_touched	Counts are derived from retro_hit; finished_at never read

Entity	Removed	Why
INCIDENT	origin	Retro-hunt incidents are already known through OPENS
ANALYST	is_active, created_at	Account housekeeping, not part of the data model

How we chose column sizes

Every text column is sized to the longest value it really needs instead of a default like 255. Fixed lists of values use ENUM, and small numbers use the smallest integer type that fits. These are the choices we thought about most.

Column	Type / Size	Why
analyst.email	VARCHAR(50)	Fits a normal address like name.surname@organisation.com, and a CHECK makes sure it looks like an email
full_name, actor_name, malware_name	VARCHAR(50)	Names stay well under 50 characters
asset.hostname	VARCHAR(63)	63 is the longest single DNS label
ip_address, src_ip	VARCHAR(45)	MySQL has no IP type; 45 characters fits the longest IPv6 address in text form
vulnerability.cve_id	VARCHAR(16)	"CVE-", a 4 digit year, "-" and up to 7 digits
attack_type.mitre_technique	VARCHAR(9)	The longest form is T1234.001
ioc.ioc_value	VARCHAR(255)	Has to hold a full domain name, which can be 253 characters
origin_country, src_country	CHAR(2)	Two letter country code, checked to be two capital letters
malware.sha256	CHAR(64)	A SHA-256 hash is always 64 hex characters
asset_type, motive, protocol, action, role, priority, status	ENUM	Only the listed values can be stored, and each takes 1 byte
criticality, severity, step_no	TINYINT	Small ranges like 1 to 10 fit in 1 byte
network_log.dst_port	SMALLINT UNSIGNED	Holds exactly 0 to 65535 in 2 bytes, so no extra check is needed
incident.title	VARCHAR(100)	Titles like "Brute Force on web-01 from 198.51.100.7" stay under 80

Tables

For each table below we give a short description, the CREATE TABLE statement for MySQL 8.0, and its normal form. The full script, with the tables in the order they must be created, is in sentinelx_mysql.sql.

Assets, vulnerabilities and logs

asset

Every server or device we protect. asset_id is the primary key, and hostname and ip_address are unique too. asset_type can only take one of the listed values, and criticality goes from 1 to 5.

```
mysql  (MySQL 8.0)   -   sentinelx
mysql> CREATE TABLE asset (
    ->   asset_id     INT AUTO_INCREMENT PRIMARY KEY,
    ->   hostname     VARCHAR(63) NOT NULL UNIQUE,           -- 63 = max DNS label
    ->   ip_address   VARCHAR(45) NOT NULL UNIQUE,           -- 45 = longest IPv6 text
    ->   asset_type   ENUM('web','db','mail','dns','workstation','iot','vpn') NOT NULL,
    ->   criticality  TINYINT NOT NULL CHECK (criticality BETWEEN 1 AND 5)
    -> );
mysql>
```

CREATE TABLE asset in MySQL

Normal form: Every column depends only on asset_id (hostname and ip_address are also candidate keys), and no non-key column depends on another. 3NF.

vulnerability

Published CVEs. cve_id is the primary key and must look like CVE-2024-3400. Severity is not stored: it follows from cvss, so the view v_vulnerability works it out when a page needs it.

```
mysql  (MySQL 8.0)   -   sentinelx
mysql> CREATE TABLE vulnerability (
    ->   cve_id     VARCHAR(16) PRIMARY KEY CHECK (cve_id REGEXP '^CVE-[0-9]{4}-[0-9]{4,7}$'),
    ->   cvss       DECIMAL(3,1) NOT NULL CHECK (cvss BETWEEN 0 AND 10),
    ->   published  DATE NOT NULL
    -> );
mysql>
```

CREATE TABLE vulnerability in MySQL

Normal form: cvss and published depend only on cve_id. Storing severity would add the transitive dependency cve_id → cvss → severity, so it was removed. 3NF.

asset_vulnerability

The link table for the many-to-many relationship EXPOSED_TO between assets and CVEs. The primary key is (asset_id, cve_id). It also stores when the CVE was found on that asset and when it was patched, and patched_at cannot be earlier than detected_at.

```
mysql  (MySQL 8.0)   -   sentinelx
mysql> CREATE TABLE asset_vulnerability (
    ->   asset_id     INT NOT NULL,
    ->   cve_id       VARCHAR(16) NOT NULL,
    ->   detected_at  DATE NOT NULL,
    ->   patched_at   DATE,
    ->   PRIMARY KEY (asset_id, cve_id),
    ->   FOREIGN KEY (asset_id) REFERENCES asset(asset_id) ON DELETE CASCADE,
    ->   FOREIGN KEY (cve_id) REFERENCES vulnerability(cve_id),
    ->   CHECK (patched_at IS NULL OR patched_at >= detected_at)
    -> );
mysql>
```

CREATE TABLE asset_vulnerability in MySQL

Normal form: detected_at and patched_at depend on the whole key, not on asset_id or cve_id alone. 3NF.

network_log

Raw log lines from the firewall and IDS. dst_asset_id must point to an existing asset. The index on (src_ip, log_time) lets a retro-hunt find the old lines for one IP in a time window without reading the whole table.

```
mysql  (MySQL 8.0)   -   sentinelx
mysql> CREATE TABLE network_log (
    ->   log_id        BIGINT AUTO_INCREMENT PRIMARY KEY,
    ->   log_time      DATETIME NOT NULL,
    ->   src_ip        VARCHAR(45) NOT NULL,
    ->   dst_asset_id  INT NOT NULL,
    ->   dst_port      SMALLINT UNSIGNED NOT NULL,          -- 0 to 65535
    ->   protocol      ENUM('TCP','UDP','ICMP') NOT NULL,
    ->   action        ENUM('allowed','blocked') NOT NULL,
    ->   FOREIGN KEY (dst_asset_id) REFERENCES asset(asset_id),
    ->   INDEX idx_src_time (src_ip, log_time)              -- used by retro-hunt
    -> );
mysql>
```

CREATE TABLE network_log in MySQL

Normal form: Everything depends on log_id. 3NF.

Attacks and attack types

attack_type

Kinds of attack, each with its MITRE ATT&CK technique ID such as T1110. type_name and mitre_technique are both unique, and base_severity is between 1 and 10.

```
mysql  (MySQL 8.0)   -   sentinelx
mysql> CREATE TABLE attack_type (
    ->   type_id          INT AUTO_INCREMENT PRIMARY KEY,
    ->   type_name        VARCHAR(40) NOT NULL UNIQUE,
    ->   mitre_technique  VARCHAR(9) NOT NULL UNIQUE
    ->     CHECK (mitre_technique REGEXP '^T[0-9]{4}([.][0-9]{3})?$'),
    ->   base_severity    TINYINT NOT NULL CHECK (base_severity BETWEEN 1 AND 10)
    -> );
mysql>
```

CREATE TABLE attack_type in MySQL

Normal form: Everything depends on type_id, and the other two unique columns are candidate keys as well. 3NF.

mitigation

Numbered response steps for each attack type. This is our weak entity, so the primary key is (type_id, step_no). If an attack type is deleted, its steps are deleted with it.

```
mysql  (MySQL 8.0)   -   sentinelx
mysql> CREATE TABLE mitigation (
    ->   type_id  INT NOT NULL,
    ->   step_no  TINYINT NOT NULL CHECK (step_no > 0),
    ->   action   VARCHAR(100) NOT NULL,
    ->   PRIMARY KEY (type_id, step_no),                    -- weak entity key
    ->   FOREIGN KEY (type_id) REFERENCES attack_type(type_id) ON DELETE CASCADE
    -> );
mysql>
```

CREATE TABLE mitigation in MySQL

Normal form: action depends on the full key (type_id, step_no). 3NF.

attack_event

One row for each detected attack. Every attack must point to an asset and an attack type. The attacker, malware and CVE are optional because many attacks are never linked to them. The indexes on event_time and src_ip serve the dashboard and the attack search.

```
mysql  (MySQL 8.0)   -   sentinelx
mysql> CREATE TABLE attack_event (
    ->   event_id           BIGINT AUTO_INCREMENT PRIMARY KEY,
    ->   event_time         DATETIME NOT NULL,
    ->   src_ip             VARCHAR(45) NOT NULL,
    ->   src_country        CHAR(2) NOT NULL CHECK (src_country REGEXP '^[A-Z]{2}$'),
    ->   asset_id           INT NOT NULL,
    ->   type_id            INT NOT NULL,
    ->   actor_id           INT,
    ->   malware_id         INT,
    ->   cve_id             VARCHAR(16),
    ->   severity           TINYINT NOT NULL CHECK (severity BETWEEN 1 AND 10),
    ->   blocked            BOOLEAN NOT NULL,
    ->   predicted_severity  ENUM('LOW','MEDIUM','HIGH','CRITICAL'),    -- filled by the ML job
    ->   FOREIGN KEY (asset_id) REFERENCES asset(asset_id),
    ->   FOREIGN KEY (type_id) REFERENCES attack_type(type_id),
    ->   FOREIGN KEY (actor_id) REFERENCES threat_actor(actor_id),
    ->   FOREIGN KEY (malware_id) REFERENCES malware(malware_id),
    ->   FOREIGN KEY (cve_id) REFERENCES vulnerability(cve_id),
    ->   INDEX idx_time (event_time),
    ->   INDEX idx_src (src_ip)
    -> );
mysql>
```

CREATE TABLE attack_event in MySQL

Normal form: Everything depends on event_id. src_country is stored here because the location of an IP can change over time, so it belongs to the attack and not to the IP. actor_id also stays, because the same malware can be used by more than one group, so malware_id does not decide actor_id. 3NF.

Attackers, malware, indicators and retro-hunts

threat_actor

Attacker groups with their country and motive. actor_name is unique. The names in our data are made up.

```
mysql  (MySQL 8.0)   -   sentinelx
mysql> CREATE TABLE threat_actor (
    ->   actor_id        INT AUTO_INCREMENT PRIMARY KEY,
    ->   actor_name      VARCHAR(50) NOT NULL UNIQUE,
    ->   origin_country  CHAR(2) CHECK (origin_country REGEXP '^[A-Z]{2}$'),
    ->   motive          ENUM('financial','espionage','hacktivism','sabotage','unknown') NOT NULL
    -> );
mysql>
```

CREATE TABLE threat_actor in MySQL

Normal form: Everything depends on actor_id. 3NF.

malware

Malware samples. malware_name and sha256 are unique, and sha256 must be 64 hex characters. actor_id is optional and becomes NULL if the attacker group is deleted. cluster_id is filled in by the ML grouping job.

```
mysql  (MySQL 8.0)   -   sentinelx
mysql> CREATE TABLE malware (
    ->   malware_id    INT AUTO_INCREMENT PRIMARY KEY,
    ->   malware_name  VARCHAR(50) NOT NULL UNIQUE,
    ->   family        VARCHAR(30) NOT NULL,
    ->   sha256        CHAR(64) NOT NULL UNIQUE CHECK (sha256 REGEXP '^[0-9a-f]{64}$'),
    ->   actor_id      INT,
    ->   cluster_id    SMALLINT,                          -- filled by the ML job
    ->   FOREIGN KEY (actor_id) REFERENCES threat_actor(actor_id) ON DELETE SET NULL
    -> );
mysql>
```

CREATE TABLE malware in MySQL

Normal form: Everything depends on malware_id. category was removed because family decides it, which was a transitive dependency. 3NF.

ioc

Indicators of compromise such as a bad IP or domain. Each row is one version. In the ER diagram valid_period (when the indicator was hostile) and recorded_period (when we knew about it) are single attributes; MySQL has no range type, so each is stored as a from/to pair, where a NULL end means the period is still open. A trigger rejects a new version that overlaps an existing one and blocks deletes, so the history cannot be lost.

```
mysql  (MySQL 8.0)   -   sentinelx
mysql> CREATE TABLE ioc (
    ->   ioc_version_id  BIGINT AUTO_INCREMENT PRIMARY KEY,
    ->   ioc_type        ENUM('ip','domain','hash','url') NOT NULL,
    ->   ioc_value       VARCHAR(255) NOT NULL,              -- 253 = longest domain name
    ->   actor_id        INT,
    ->   malware_id      INT,
    ->   valid_from      DATETIME NOT NULL,                  -- valid_period: when it was hostile
    ->   valid_to        DATETIME,
    ->   recorded_from   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ->   -- recorded_period: when we knew
    ->   recorded_to     DATETIME,
    ->   FOREIGN KEY (actor_id) REFERENCES threat_actor(actor_id),
    ->   FOREIGN KEY (malware_id) REFERENCES malware(malware_id),
    ->   CHECK (ioc_type <> 'ip' OR ioc_value REGEXP '^[0-9]{1,3}([.][0-9]{1,3}){3}$'),
    ->   CHECK (valid_to IS NULL OR valid_to > valid_from),
    ->   CHECK (recorded_to IS NULL OR recorded_to > recorded_from),
    ->   INDEX idx_value (ioc_value)
    -> );
mysql>
```

CREATE TABLE ioc in MySQL

Normal form: Everything depends on ioc_version_id. 3NF.

event_ioc

The link table for the many-to-many relationship MATCHES between attacks and indicators.

```
mysql  (MySQL 8.0)   -   sentinelx
mysql> CREATE TABLE event_ioc (
    ->   event_id        BIGINT NOT NULL,
    ->   ioc_version_id  BIGINT NOT NULL,
    ->   PRIMARY KEY (event_id, ioc_version_id),
    ->   FOREIGN KEY (event_id) REFERENCES attack_event(event_id) ON DELETE CASCADE,
    ->   FOREIGN KEY (ioc_version_id) REFERENCES ioc(ioc_version_id)
    -> );
mysql>
```

CREATE TABLE event_ioc in MySQL

Normal form: The whole row is the key. 3NF.

retro_hunt_run

One row each time the database searches old logs for a new bad IP. Every run must point to the indicator that started it. incident_id is unique, so a run can open at most one incident. The number of matched logs and touched assets comes from the view v_retro_hunt_summary.

```
mysql  (MySQL 8.0)   -   sentinelx
mysql> CREATE TABLE retro_hunt_run (
    ->   run_id          INT AUTO_INCREMENT PRIMARY KEY,
    ->   ioc_version_id  BIGINT NOT NULL,
    ->   started_at      DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ->   incident_id     INT UNIQUE,                        -- at most one incident per run
    ->   FOREIGN KEY (ioc_version_id) REFERENCES ioc(ioc_version_id),
    ->   FOREIGN KEY (incident_id) REFERENCES incident(incident_id)
    -> );
mysql>
```

CREATE TABLE retro_hunt_run in MySQL

Normal form: Everything depends on run_id. The search window is not stored because the indicator already decides it, and the counts are not stored because retro_hit already holds them. 3NF.

retro_hit

The link table for the many-to-many relationship HITS: the old log lines a retro-hunt found.

```
mysql  (MySQL 8.0)   -   sentinelx
mysql> CREATE TABLE retro_hit (
    ->   run_id  INT NOT NULL,
    ->   log_id  BIGINT NOT NULL,
    ->   PRIMARY KEY (run_id, log_id),
    ->   FOREIGN KEY (run_id) REFERENCES retro_hunt_run(run_id) ON DELETE CASCADE,
    ->   FOREIGN KEY (log_id) REFERENCES network_log(log_id)
    -> );
mysql>
```

CREATE TABLE retro_hit in MySQL

Normal form: The whole row is the key. 3NF.

Incidents and analysts

analyst

People who use the system. email is unique, at most 50 characters and checked to look like an email address. role can only be admin, analyst or viewer.

```
mysql  (MySQL 8.0)   -   sentinelx
mysql> CREATE TABLE analyst (
    ->   analyst_id  INT AUTO_INCREMENT PRIMARY KEY,
    ->   full_name   VARCHAR(50) NOT NULL,
    ->   email       VARCHAR(50) NOT NULL UNIQUE CHECK (email REGEXP '^[^@ ]+@[^@ ]+[.][A-Za-z]{2,}$'),
    ->   role        ENUM('admin','analyst','viewer') NOT NULL
    -> );
mysql>
```

CREATE TABLE analyst in MySQL

Normal form: Everything depends on analyst_id, and email is also a candidate key. 3NF.

incident

An investigation. It can be opened by the trigger, by an analyst or by a retro-hunt. assigned_to is optional because a new incident has no owner yet. closed_at is filled in only when the status is CLOSED.

```
mysql  (MySQL 8.0)   -   sentinelx
mysql> CREATE TABLE incident (
    ->   incident_id  INT AUTO_INCREMENT PRIMARY KEY,
    ->   title        VARCHAR(100) NOT NULL,
    ->   priority     ENUM('LOW','MEDIUM','HIGH','CRITICAL') NOT NULL,
    ->   status       ENUM('OPEN','INVESTIGATING','CONTAINED','CLOSED') NOT NULL DEFAULT 'OPEN',
    ->   assigned_to  INT,
    ->   opened_at    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ->   closed_at    DATETIME,
    ->   FOREIGN KEY (assigned_to) REFERENCES analyst(analyst_id),
    ->   CHECK ((status = 'CLOSED') = (closed_at IS NOT NULL)),
    ->   CHECK (closed_at IS NULL OR closed_at >= opened_at)
    -> );
mysql>
```

CREATE TABLE incident in MySQL

Normal form: Everything depends on incident_id. 3NF.

incident_event

The link table for the many-to-many relationship GROUPS between incidents and attacks.

```
mysql  (MySQL 8.0)   -   sentinelx
mysql> CREATE TABLE incident_event (
    ->   incident_id  INT NOT NULL,
    ->   event_id     BIGINT NOT NULL,
    ->   PRIMARY KEY (incident_id, event_id),
    ->   FOREIGN KEY (incident_id) REFERENCES incident(incident_id) ON DELETE CASCADE,
    ->   FOREIGN KEY (event_id) REFERENCES attack_event(event_id) ON DELETE CASCADE
    -> );
mysql>
```

CREATE TABLE incident_event in MySQL

Normal form: The whole row is the key. 3NF.

9. Module-wise Functional Description

Dashboard and Attack Search

The home page shows attacks and blocked attacks in the last 24 hours, open critical incidents, malicious IPs and the most attacked servers. All of it is read from database views. The search page filters attacks by IP, country, attack type, malware, CVE, severity and date.

Incident Module

A trigger opens an incident for any attack of severity 8 or more. Attacks of the same type on the same server within one hour are added to the same incident so analysts are not flooded with duplicates. Assigning an incident goes through a stored procedure that locks the row first (SELECT … FOR UPDATE), so two analysts cannot take the same incident at once.

Retro-Hunt Module

When a new bad IP is added, a trigger works out the period when the IP was already hostile but we did not know it yet, and searches the old logs for that period. The index on (src_ip, log_time) means MySQL reads only the matching rows instead of the whole log table. In our test data, an IP that had touched 9 servers 340 times a month earlier was raised as a critical incident as soon as it was added.

Time Machine Module

Every indicator keeps both "when it was hostile" and "when we knew", so a search can be repeated as of an earlier date. Over the same 180 days the search finds 4,934 attacks with what we knew in early April and 43,373 with what we know today. 3,286 attacks were only recognised later, usually about 11 days after they happened.

Threat Intelligence, ML and Roles

Separate pages list attacker groups, malware and CVEs. Two ML jobs write their answers into existing columns: the predicted severity of an attack and the group a malware sample belongs to. There are three MySQL roles. Admins can do everything, analysts work on incidents and indicators, and viewers can only read the dashboard views.

10. Work Break Down

Roughly how the work was split, although in practice there was a lot of overlap and all three of us reviewed each other's schema changes before merging.

Reg. No.	Name	Work Assigned
24BCI0061	Shlok Gupta	Retro-hunt trigger and indicator history design, indexes, FastAPI backend, Docker setup
24BCI0086	Shreya Bhushan	ER diagram, normalisation, table creation and constraints, views and stored procedures, user roles, this report
24BDS0158	Yashneil	Test data generator, ML models, React dashboard, testing

11. Limitations and Future Scope

A few things we would do differently or add if this went beyond a lab project:

- The data is generated, not real. Attacker IPs come from address ranges reserved for documentation so no real address is marked as bad, and attacker and malware names are made up. The CVE IDs are real.
- Retro-hunts only work for IP addresses for now. Domains and file hashes would need DNS and file logs.
- MySQL has no range type or exclusion constraint, so the rule that two versions of an indicator never overlap is checked by a trigger instead of a declared constraint.
- The ML columns hold simple starting values until the models are fully trained.
- Logs are loaded in batches. Very high volumes would need a streaming pipeline such as Kafka, and old logs would need to move to archive tables.

12. Conclusion

Working on SentinelX made the usual database topics feel a lot less abstract. Normalisation stopped being an exercise once we found two transitive dependencies hiding in our own first draft, severity depending on cvss and category depending on family. Reviewing our ER diagram also taught us to keep only the attributes a feature really uses, which cut 23 columns and made the design much easier to read. The two parts we like most, retro-hunting and the time machine, turned out to be a trigger, two pairs of date columns and one index, which is a good reminder that careful table design does most of the work in a system like this.

REVIEW EVALUATION

COMPONENT	MARKS	SHLOK GUPTA	SHREYA BHUSHAN	YASHNEIL
ER-diagram	3			
Table Creation with description and screenshot	5			
Normal form	2			
TOTAL	**10 MARKS**			